

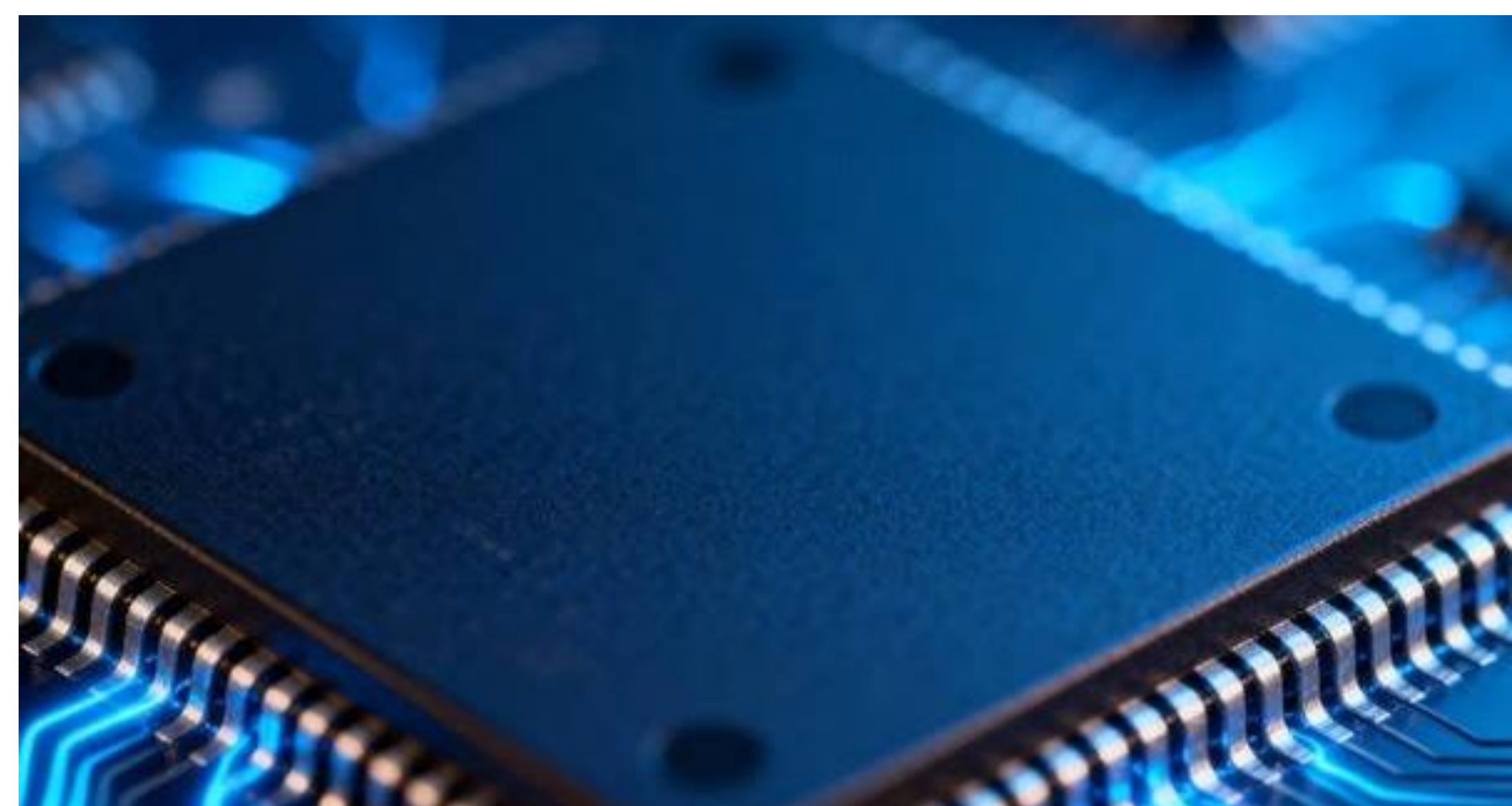
Détection d'Anomalies du code VHDL par Réseau de Neurones Récurrent (RNN)

Réalisé par BOUDADE Ilyass – ATIK Zakaria – DOUIBA Youssef
Encadré par Pr. HANNOUI Salma – Pr. BEN LAHMAR El Habib

Introduction / Contexte

Les langages de description matérielle comme le VHDL sont les piliers des architectures numériques modernes. Les méthodes traditionnelles (compilateurs, linters) échouent à détecter les anomalies logiques ou sémantiques subtiles avant la simulation fonctionnelle active.

La problématique posée au sein des limites des méthodes traditionnelles concerne le dépassement de la simple vérification syntaxique afin d'identifier des comportements structurels défailants.



Architecture

Le modèle RNN a été entièrement implémenté de manière matricielle en **NumPy** afin de garantir une maîtrise totale des dynamiques de rétropropagation temporelle et d'éviter les abstractions des frameworks de haut niveau (ex. PyTorch)

Hyperparamètres Fixés :

Dimension d'Embedding : 64

État Caché : 128 neurones

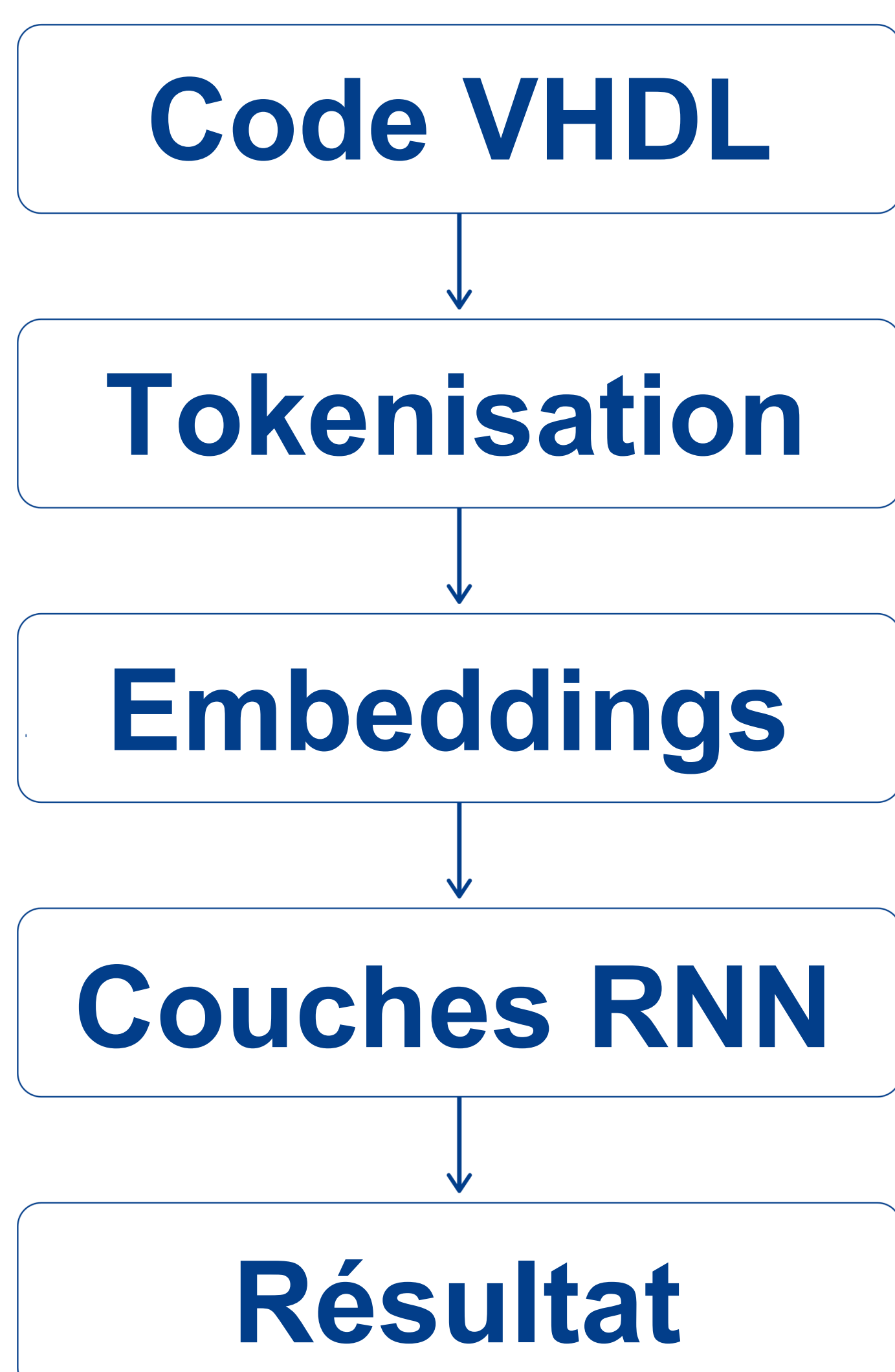
Longueur Maximale de Séquence :

512 tokens

Optimiseur :

Adam - $l_r = 3 \times 10^{-3}$

Gradient Clipping : 5.0



Traitement des données

Les données utilisés pour notre projet sont initialement pris et nettoyés d'un jeu de données standardisé depuis la plateforme Hugging Face.

Pour pallier l'absence de code defectueux réel, une procédure d'injection d'erreurs a été implémenté pour équilibrer le dataset à **8 613 échantillons** (50% sains, 50% corrompus). Cette procédure constitue de l'application de quatre stratégies:

(1) Dead Code Injection - Injection du code mort : 1154 échantillons

Injection de fausses lignes de commentaires d'erreur, des expressions invalides ou des assignations de signaux morts à intervalles réguliers, utilisant des identifiants inexistantes ou des valeurs forcées.

(2) Structural Havoc - Destruction de la structure globale :

Altération des assignations à l'intérieur de l'architecture de l'extrait.

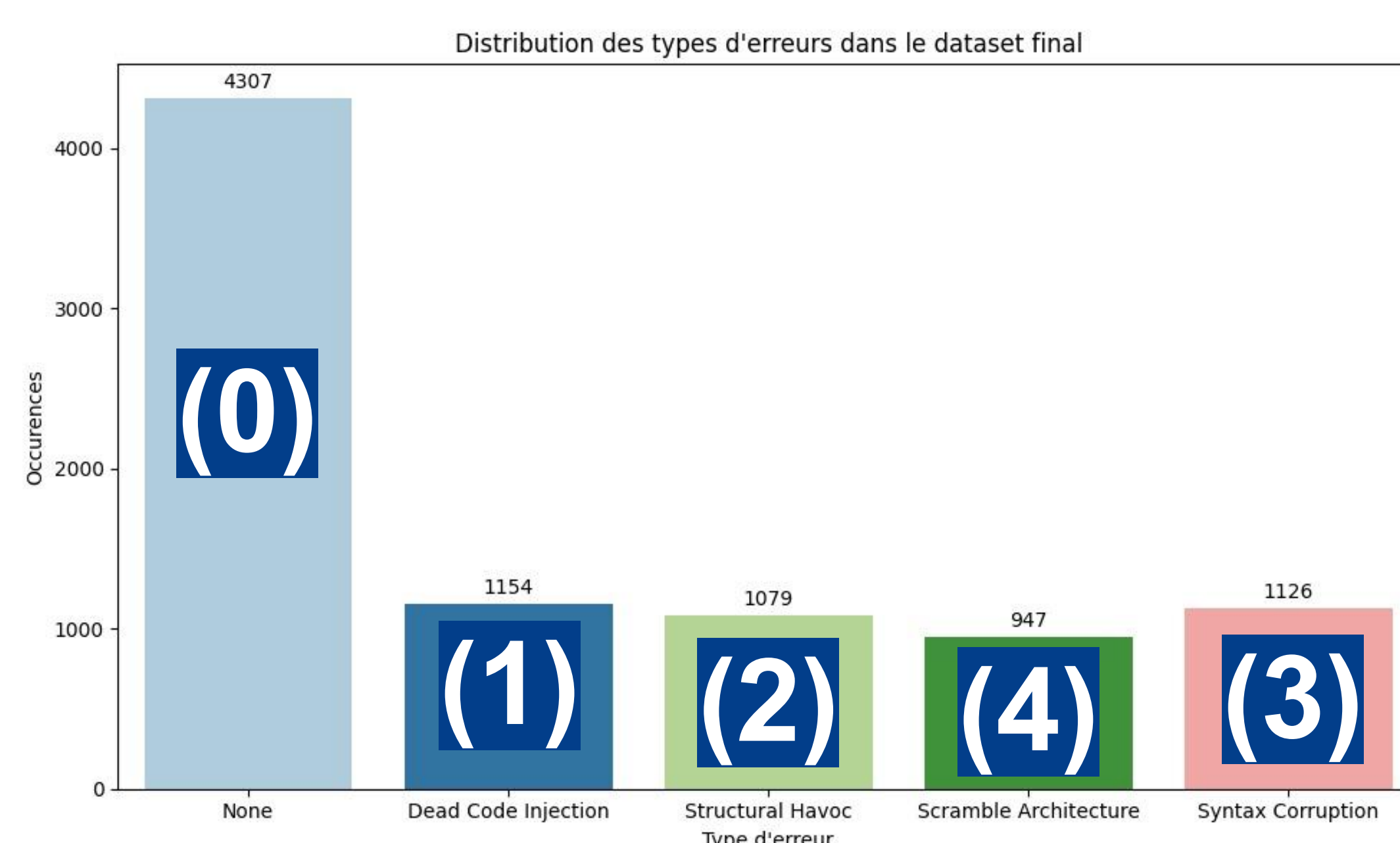
(3) Syntax Corruption - Corruption de syntax du code : 1079 échantillons

Retrait des points-virgules et remplacement des mots-clés natifs du VHDL par leurs équivalents

(4) Scramble Architecture - Créer du chaos séquentiel : 947 échantillons

Isolation de toutes les lignes constituant le cœur du traitement matériel.

Ces lignes sont ensuite mélangées de manière totalement aléatoire réorganisant de façon anarchique les assignations de signaux ou les instanciations de processus.



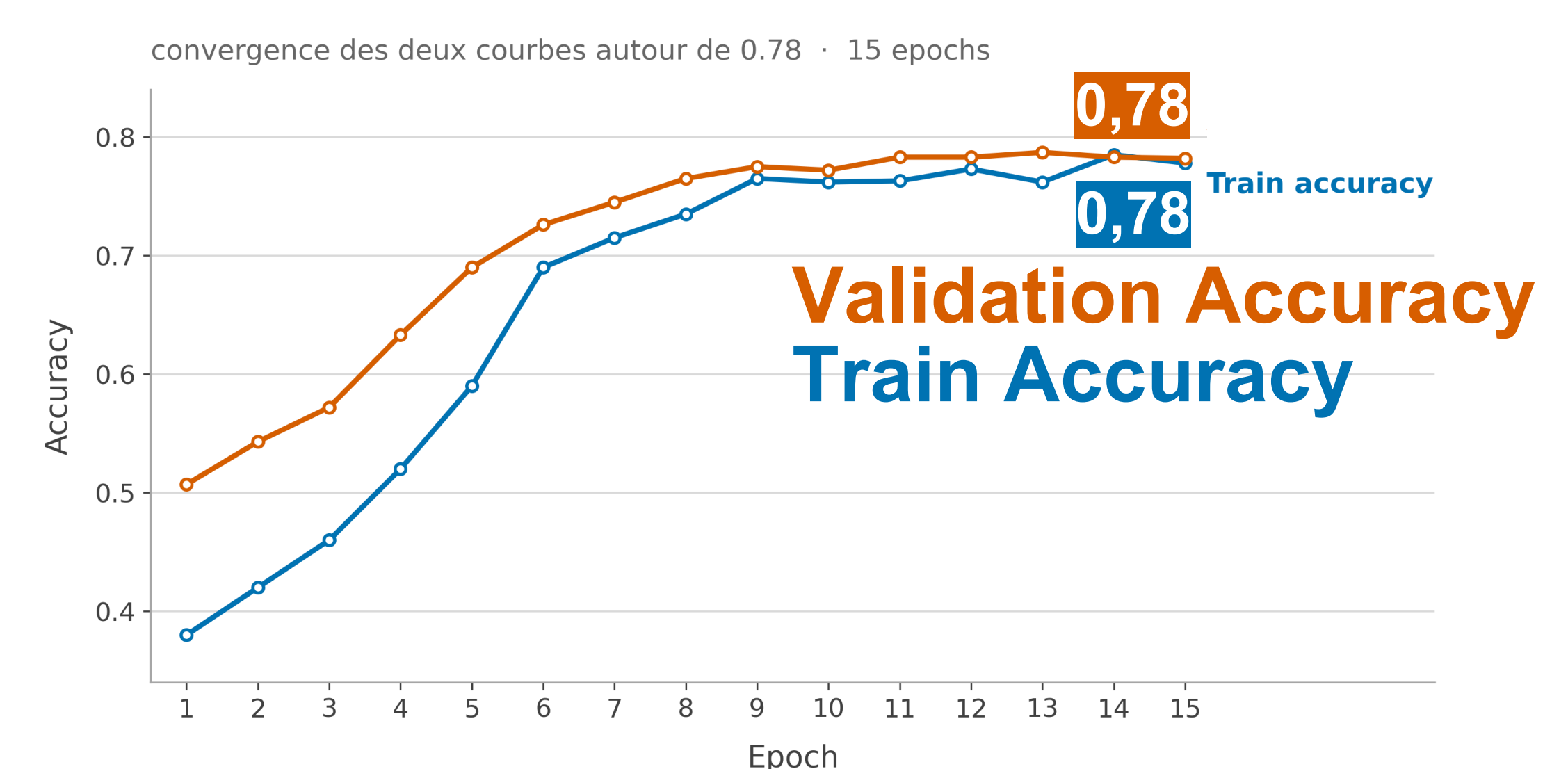
Résultats

Performances Globales

Le modèle atteint globalement une exactitude finale sur l'ensemble de test de **78,40 %**.

Les courbes révèlent une convergence rapide dès la 6ème époque, suivie d'une phase de stabilisation fine montrant un comportement robuste face aux bruits du dataset.

Accuracy curves (early learning phase)



Sensibilités aux anomalies

Anomalies Syntaxiques

Le modèle détecte les erreurs syntaxiques avec un taux approximatif de **80%**, grâce à la capture immédiate des ruptures grammaticales.

Anomalies Sémantiques

Le modèle tend à la suspicion sur certains extraits sains. Cependant, il détecte les erreurs sémantiques avec un taux approximatif de **60%**.

```
=====
TEST: NORMAL CODE
=====
Anomaly probability: 0.2146
Prediction: NORMAL ✓
```

```
=====
TEST: SYNTAX ERROR
=====
Anomaly probability: 0.7955
Prediction: ANOMALY ✗
```

```
=====
TEST: SEMANTIC ERROR
=====
Anomaly probability: 0.5453
Prediction: ANOMALY ✗
```

Conclusion et Perspectives

Ce projet nous a permis d'implémenter un réseau de neurones récurrent séquentiel pour améliorer la détection d'anomalies du code VHDL sémantiques et syntaxiques.

La voie aux axes d'amélioration est ouverte afin d'améliorer la sensibilité aux anomalies sémantiques et éliminer les faux positifs :

Transition vers des architectures **LSTM / Bi-LSTM**, Extension vers des modèles de type **Transformers**, Augmentation et rééquilibrage avancé du dataset synthétique.